Final Project: Bonus Question

Henry Dixon 05/01/20

This report provides the response to the Final Project Bonus question for ASEN 5044. The diagram for this system model is shown below in Figure 1. After presenting the combined system model information and dynamics again, the report walks through the algorithm derivation, simulation, and implementation of my Unscented Kalman Filter (UKF). As well, Monte-Carlo Truth Model performance Testing (TMT) results are shown and discussed.

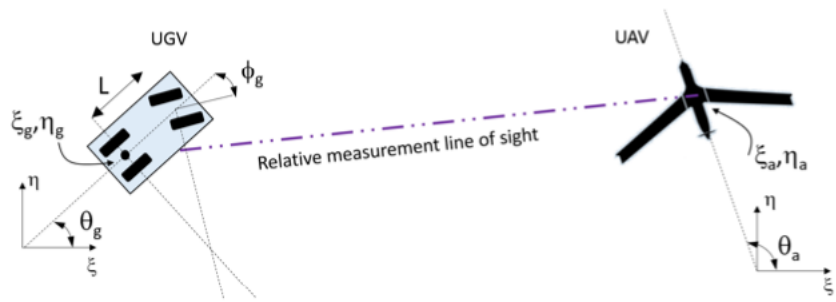

Figure 1: Cooperative air - ground localization problem setup.

1. Combined System Model Parameters

This section defines the the parameters for the cooperative UGV - UAV system, the nonlinear state space definition for the model and shows the derivation of the Jacobians used to linearize the nonlinear system. The dynamics for the UGV system and process noise are modeled by the following equations:

$$\dot{\xi}_g = v_g cos\theta_g + \tilde{w}_{x,g} \tag{1}$$

$$\dot{\eta}_g = v_g sin\theta_g + \tilde{w}_{y,g} \tag{2}$$

$$\dot{\theta}_g = \frac{v_g}{L} tan\phi_g + \tilde{w}_{\omega,g} \tag{3}$$

With control inputs:

$$\boldsymbol{u}_g(t) = \begin{bmatrix} v_g \\ \phi_g \end{bmatrix} \tag{4}$$

And process noise:

$$\tilde{\boldsymbol{w}}_g(t) = \begin{bmatrix} \tilde{w}_{x,g} \\ \tilde{w}_{y,g} \\ \tilde{w}_{\omega,g} \end{bmatrix} \tag{5}$$

In this problem, the UGV begins navigating from a nominal position and initiates a steady continuous turning maneuver. The UGV's wheel base length is $L = 0.5\ m$ and $u_2\ \epsilon\ [-\frac{5\pi}{12}, \frac{5\pi}{12}]\ rad$, and $u_{1,max} = 3\ m/s$.

Likewise, the equations of motion, including process noise, for the UAV are defined as:

$$\dot{\xi}_a = v_a cos\theta_a + \tilde{w}_{x,a} \tag{6}$$

$$\dot{\eta}_a = v_a sin\theta_a + \tilde{w}_{y,a} \tag{7}$$

$$\dot{\theta}_a = \omega_a + \tilde{w}_{\omega,a} \tag{8}$$

With control inputs:

$$\boldsymbol{u}_a(t) = \begin{bmatrix} v_a \\ \omega_a \end{bmatrix} \tag{9}$$

And process noise:

$$\tilde{\boldsymbol{w}}_a(t) = \begin{bmatrix} \tilde{w}_{x,a} \\ \tilde{w}_{y,a} \\ \tilde{w}_{\omega,a} \end{bmatrix} \tag{10}$$

The combined system state, control input, and disturbance input vectors are:

$$\boldsymbol{x}(t) = \begin{bmatrix} \xi_g \\ \eta_g \\ \theta_g \\ \xi_a \\ \eta_a \\ \theta_a \end{bmatrix} \tag{11}$$

$$\boldsymbol{u}(t) = \begin{bmatrix} \boldsymbol{u}_g \\ \boldsymbol{u}_a \end{bmatrix} \tag{12}$$

$$\tilde{\boldsymbol{w}}(t) = \begin{bmatrix} \tilde{\boldsymbol{w}}_g \\ \tilde{\boldsymbol{w}}_a \end{bmatrix} \tag{13}$$

Similar to the UGV, in this problem the aircraft begins flying from an initial position with a nominal trajectory and initiates a steady continuous turning maneuver. The UAV's turning rate is limited to $u_4\ \epsilon\ [-\frac{\pi}{6}, \frac{\pi}{6}]\ rad/s$ and flies with a velocity limited to $u_3\ \epsilon\ [10, 20]\ m/s$.

Lastly, the combined system's sensing model and nonlinear state space representation are defined as:

$$\dot{\boldsymbol{x}}(t) = \begin{bmatrix} u_1 cos x_3 \\ u_1 sin x_3 \\ \frac{u_1}{L} tan u_2 \\ u_3 cos x_6 \\ u_3 sin x_6 \\ u_4 \end{bmatrix} = \boldsymbol{f}(x,u) + \boldsymbol{\Gamma}\tilde{\boldsymbol{w}}(t) \tag{14}$$

$$\boldsymbol{y}(t) = \begin{bmatrix} tan^{-1}\frac{x_5-x_2}{x_4-x_1} - x_3 \\ \sqrt{(x_1-x_4)^2+(x_2-x_5)^2} \\ tan^{-1}\frac{x_2-x_5}{x_1-x_4} - x_6 \\ x_4 \\ x_5 \end{bmatrix} + \tilde{\boldsymbol{v}}(t) = \boldsymbol{h}(x) + \tilde{\boldsymbol{v}}(t) \tag{15}$$

where $\tilde{\boldsymbol{v}}(t)$ is the AWGN sensor error vector.

2. Unscented Kalman Filter Algorithm Derivation

In the final project report we saw how Kalman Filter performance improved between the implementations of the Linearized and Extended Kalman Filters. When perturbation inputs to the model grew too large, the LKF began to struggle in maintaining accurate estimate tracking of the ground truth. The EKF was able to perform better and recover from larger perturbations because of the live online trajectory update at the beginning of each prediction step and full state estimate at the end of every correction step - instead of just an estimate of perturbation from an a-priori nominal trajectory. The Unscented Kalman Filter will take the EKF enhancements one step further to achieve better performance.

The UKF is a fully nonlinear filter; formulated by applying an Unscented Transform to the model dynamics, since it is easier to directly approximate mean and covariance between pdf's than through the nonlinear state transition functions. Instead of using a Taylor Series expansion to linearize the model dynamics, sigma points are generated from the current state pdf $p(X_k \,|\, Y_{1:k})$ and propagated to the next time step through the nonlinear dynamics to approximate the first two moments of the pdf at the future time step. Where $2n + 1$ sigma points are required to accurately sketch out $\boldsymbol{E}[\boldsymbol{x}_k] = \boldsymbol{\mu}_x$ and $cov(\boldsymbol{x}_k) = \boldsymbol{P}_{xx}$ from $\boldsymbol{x}_k \sim p(x_k|y_{1:k})$. Given that $\boldsymbol{P}_{xx} = \boldsymbol{S}^T\boldsymbol{S}$ and $\boldsymbol{S} = chol(\boldsymbol{P}_{xx})$, sigma points are computed as:

$$\boldsymbol{\chi}^0 = \boldsymbol{\mu}_x \tag{16}$$

$$\boldsymbol{\chi}^i = \begin{cases} \boldsymbol{\mu}_x + (\sqrt{n+\lambda}) \cdot \boldsymbol{S}^{j,T}, & for\ i = 1,...,n,\ and\ j = 1,...,n \\ \boldsymbol{\mu}_x - (\sqrt{n+\lambda}) \cdot \boldsymbol{S}^{j,T}, & for\ i = n+1,...,2n,\ and\ j = 1,...,n \end{cases} \tag{17}$$

Where n is the number of states in $\boldsymbol{x}_k$ and λ is the scaling parameter defined by:

$$\lambda = \alpha^2 \cdot (n + k) - n \tag{18}$$

The propagated sigma points can then be recombined and used to estimate the new mean and covariance. The variable α controls the spread of the sigma points and as its magnitude is reduced, the estimate performance approaches that of the EKF. Now, with the unscented transformation process defined, we can begin the derivation of the UKF algorithm for additive process and measurement noise.

Like the LKF and EKF algorithms, presented in the final project report, the UKF algorithm incorporates a prediction step and measurement update/correction step. However, a nonlinear transformation will be performed at both steps - approximating the statistics using the generated sigma points. The prediction step is computed as follows, given initial values for $\hat{\boldsymbol{x}}_k^+$, $\boldsymbol{P}_k^+$, and $\boldsymbol{S}_k$:

$$\boldsymbol{\chi}_k^0 = \hat{\boldsymbol{x}}_k^+ \tag{19}$$

$$\boldsymbol{\chi}^i = \begin{cases} \hat{\boldsymbol{x}}^+ + (\sqrt{n+\lambda}) \cdot \boldsymbol{S}_k^{j,T}, & for\ i = 1, ..., n,\ and\ j = 1, ..., n \\ \hat{\boldsymbol{x}}^+ - (\sqrt{n+\lambda}) \cdot \boldsymbol{S}_k^{j,T}, & for\ i = n+1, ..., 2n,\ and\ j = 1, ..., n \end{cases} \tag{20}$$

Then each of the sigma points at $t = t_k$ is propagated through the nonlinear dynamics function $\boldsymbol{f}$ to generate predicted points ($\boldsymbol{\chi}_{k+1}^{-0}$ *and* $\boldsymbol{\chi}_{k+1}^{-i}$ at $t = t_{k+1}$. The prediction step is concluded by recombining the resultant points to compute the predicted mean and covariance by:

$$\hat{\boldsymbol{x}}_{k+1}^- \approx \sum_{i=0}^{2n} \boldsymbol{\omega}_m^i \cdot \boldsymbol{\chi}_{k+1}^{-i} \tag{21}$$

$$\boldsymbol{P}_{k+1}^- \approx \sum_{i=0}^{2n} \boldsymbol{\omega}_m^i \cdot (\boldsymbol{\chi}_{k+1}^{-i} - \hat{\boldsymbol{x}}_{k+1}^-)(\boldsymbol{\chi}_{k+1}^{-i} - \hat{\boldsymbol{x}}_{k+1}^-)^T + \boldsymbol{Q}_k \tag{22}$$

Where:

$$\omega_m^0 = \frac{\lambda}{n+\lambda}$$

$$\omega_c^0 = \frac{\lambda}{n+\lambda} + 1 - \alpha^2 + \beta$$

$$\omega_m^i = \frac{1}{2(n+\lambda)} = \omega_c^i$$

The measurement update step has a form similar to the prediction step. Given $\hat{\boldsymbol{x}}^-_{k+1}$, $\boldsymbol{P}^-_{k+1}$, $\bar{\boldsymbol{S}}_{k+1}$, and observation $\boldsymbol{y}_{k+1}$, the measurement update step begins by generating another set of sigma points and the step procedure is as follows:

$$\boldsymbol{\chi}^0_{k+1} = \hat{\boldsymbol{x}}^-_{k+1} \tag{23}$$

$$\boldsymbol{\chi}^i_{k+1} = \begin{cases} \hat{\boldsymbol{x}}^-_{k+1} + (\sqrt{n+\lambda}) \cdot \bar{\boldsymbol{S}}^{j,T}_{k+1}, & for\ i = 1, ..., n,\ and\ j = 1, ..., n \\ \hat{\boldsymbol{x}}^-_{k+1} - (\sqrt{n+\lambda}) \cdot \bar{\boldsymbol{S}}^{j,T}_{k+1}, & for\ i = n+1, ..., 2n,\ and\ j = 1, ..., n \end{cases} \tag{24}$$

Then the sigma points at $t = t_k + 1$ are propagated through the nonlinear sensor dynamics function $\boldsymbol{h}$ to generate resultant points ($\boldsymbol{\gamma}^0_{k+1}$ *and* $\boldsymbol{\gamma}^i_{k+1}$. After solving for the new sigma points for the sensor dynamics, the updated measurement mean and covariance prediction is computed by:

$$\hat{\boldsymbol{y}}^-_{k+1} \approx \sum_{i=0}^{2n} \boldsymbol{\omega}^i_m \cdot \boldsymbol{\gamma}^i_{k+1} \tag{25}$$

$$\boldsymbol{P}^-_{yy,k+1k+1} \approx \sum_{i=0}^{2n} \boldsymbol{\omega}^i_m \cdot (\boldsymbol{\gamma}^i_{k+1} - \hat{\boldsymbol{y}}^-_{k+1})(\boldsymbol{\gamma}^i_{k+1} - \hat{\boldsymbol{y}}^-_{k+1})^T + \boldsymbol{R}_{k+1} \tag{26}$$

Next, the state-measurement cross-covariance and Kalman gains are calculated in order to finally perform the update for the Kalman state and and covariance using the observation $\boldsymbol{y}_{k+1}$.

$$\boldsymbol{P}_{xy,k+1} \approx \sum_{i=0}^{2n} \boldsymbol{\omega}^i_c \cdot (\boldsymbol{\chi}^{-i}_{k+1} - \hat{\boldsymbol{x}}^-_{k+1})(\boldsymbol{\gamma}^i_{k+1} - \hat{\boldsymbol{y}}^-_{k+1})^T \tag{27}$$

$$\boldsymbol{K}_{k+1} \approx \boldsymbol{P}_{xy,k+1} \cdot [\boldsymbol{P}_{yy,k+1}]^{-1} \tag{28}$$

$$\hat{\boldsymbol{x}}^+_{k+1} = \hat{\boldsymbol{x}}^-_{k+1} + \boldsymbol{K}_{k+1}\boldsymbol{P}_{yy,k+1}\boldsymbol{K}^T_{k+1} = \boldsymbol{P}^-_{k+1} - \boldsymbol{P}_{xy,k+1}[\boldsymbol{P}_{yy,k+1}]^{-1}\boldsymbol{P}^T_{xy,k+1} \tag{29}$$

At the end of the measurement update step, the recursive filter returns to the beginning of the prediction step. With the algorithm defined, the next section will discuss the initial and tuned filter implementation results.

3. UKF Simulation Results

As was done for the LKF and EKF in the final project report, initial simulations of the derived algorithm were performed. This was done as an initial check to make sure the filter algorithm was programmed correctly - before tuning the filter via Truth Model Testing (TMT). The initial filter simulation was performed by initializing the UKF with $\boldsymbol{P}_0 = 0.9 \cdot diag([20,\ 20,\ 2,\ 20\ 20\ \ 2])$ and $\hat{\boldsymbol{x}}_0 = \boldsymbol{\mu}_0$. The filter's state estimate error and estimate tracking compared to ground truth are displayed in Figure 2 and Figure 3.

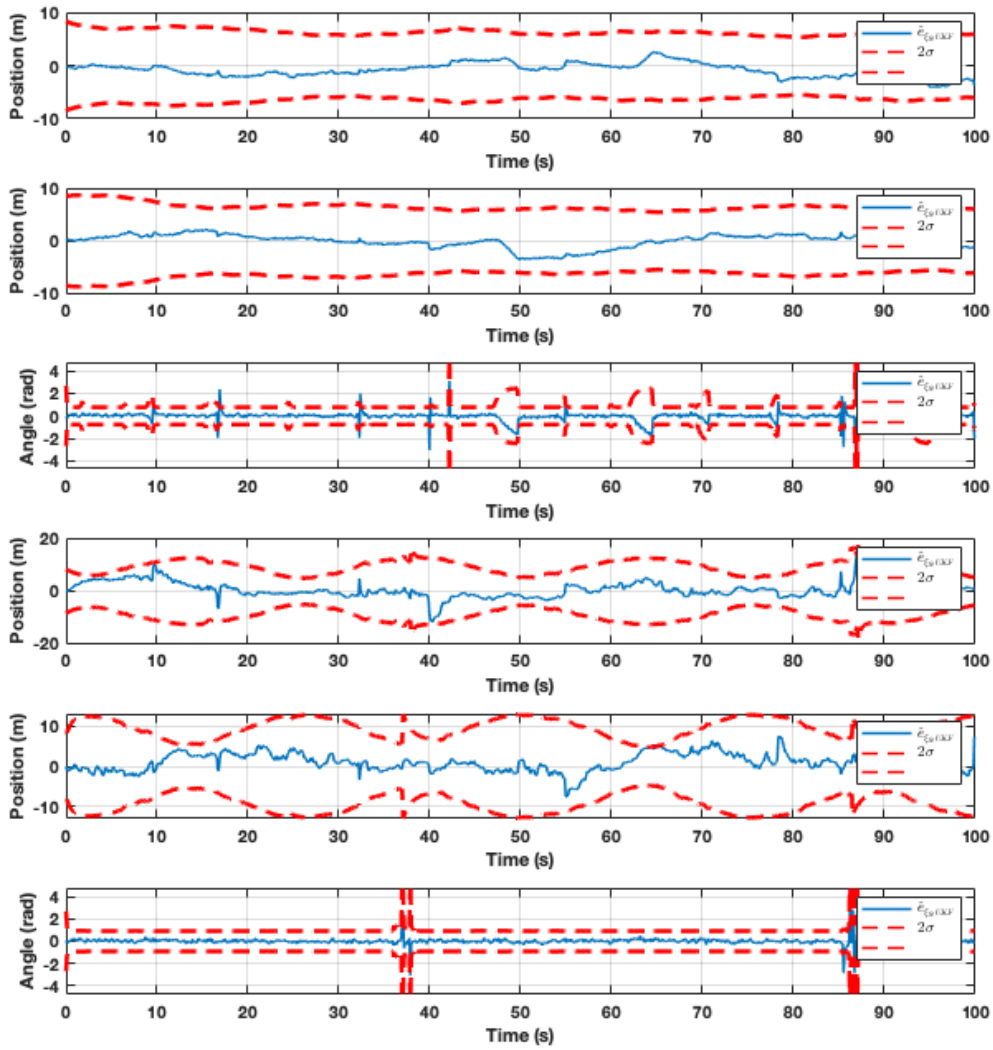

Figure 2: Initial UKF state estimate error and 2σ bounds.

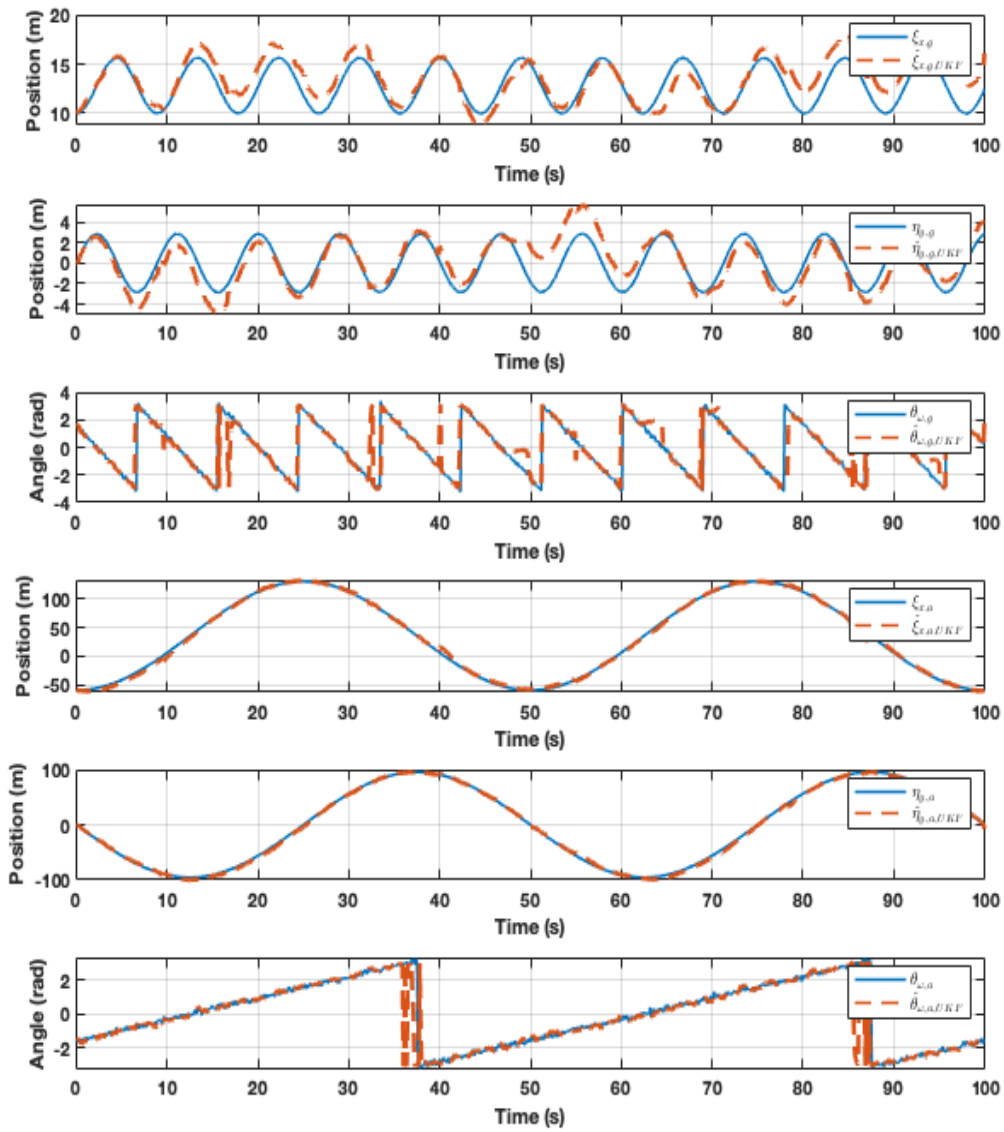

Figure 3: Initial UKF state estimate tracking comparison with ground truth.

The initial results for the estimator errors a performance improvement from the the initial LKF simulation and performance similar to the initial EKF simulation. This is more easily observable in the tracking performance results plot. Figures 4 and 5 show simulations results from the tuned filter.

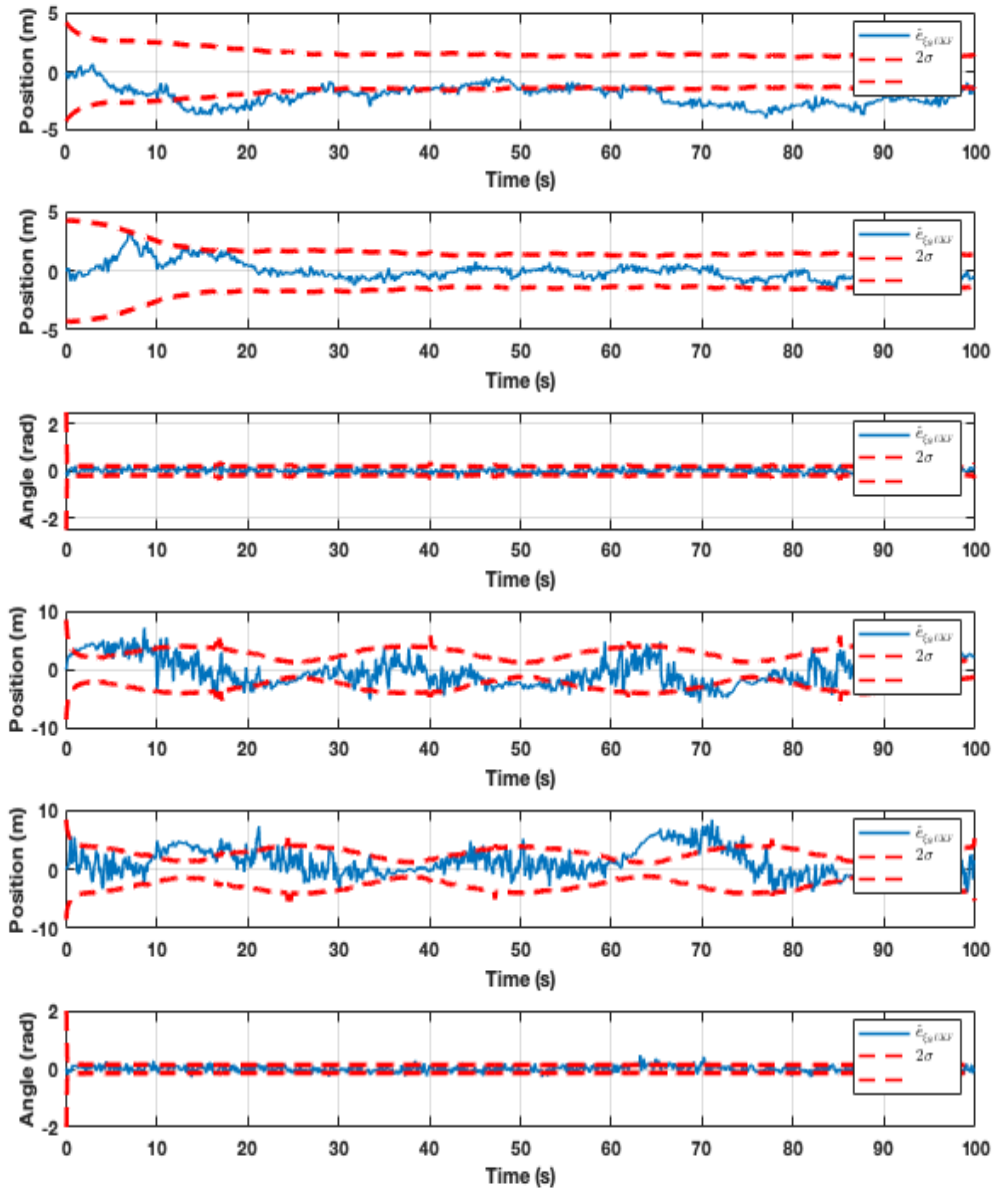

Figure 4: Tuned UKF state estimate error and 2σ bounds.

We can see that the state estimate error and uncertainty bounds are much improved after tuning the filter through the TMT process. The reduction in state estimate error

is can be even more easily observed in the tracking comparison plot. The measurement residuals from the simulation are shown in Figure 6.

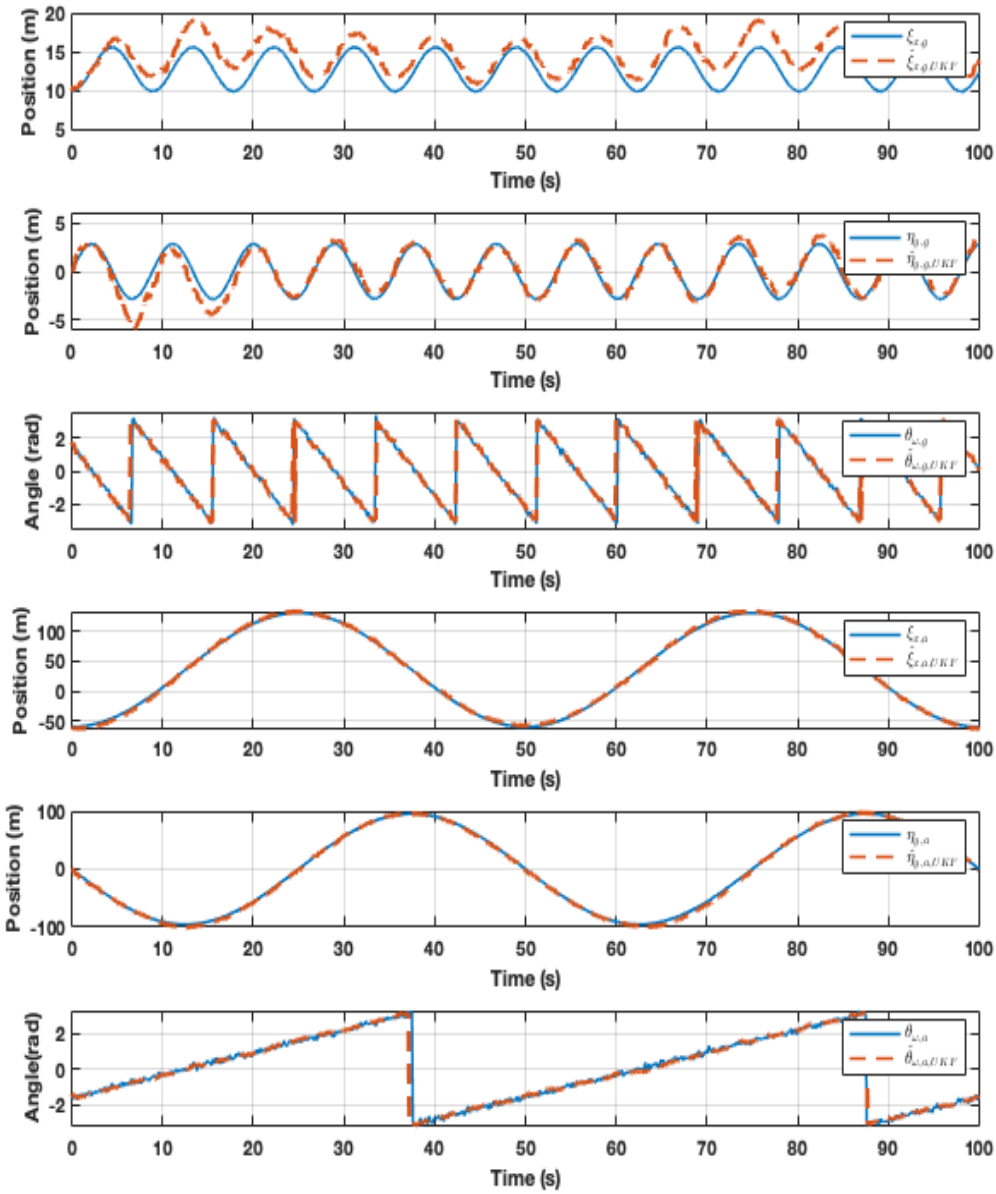

Figure 5: Tuned UKF state estimate tracking comparison with ground truth.

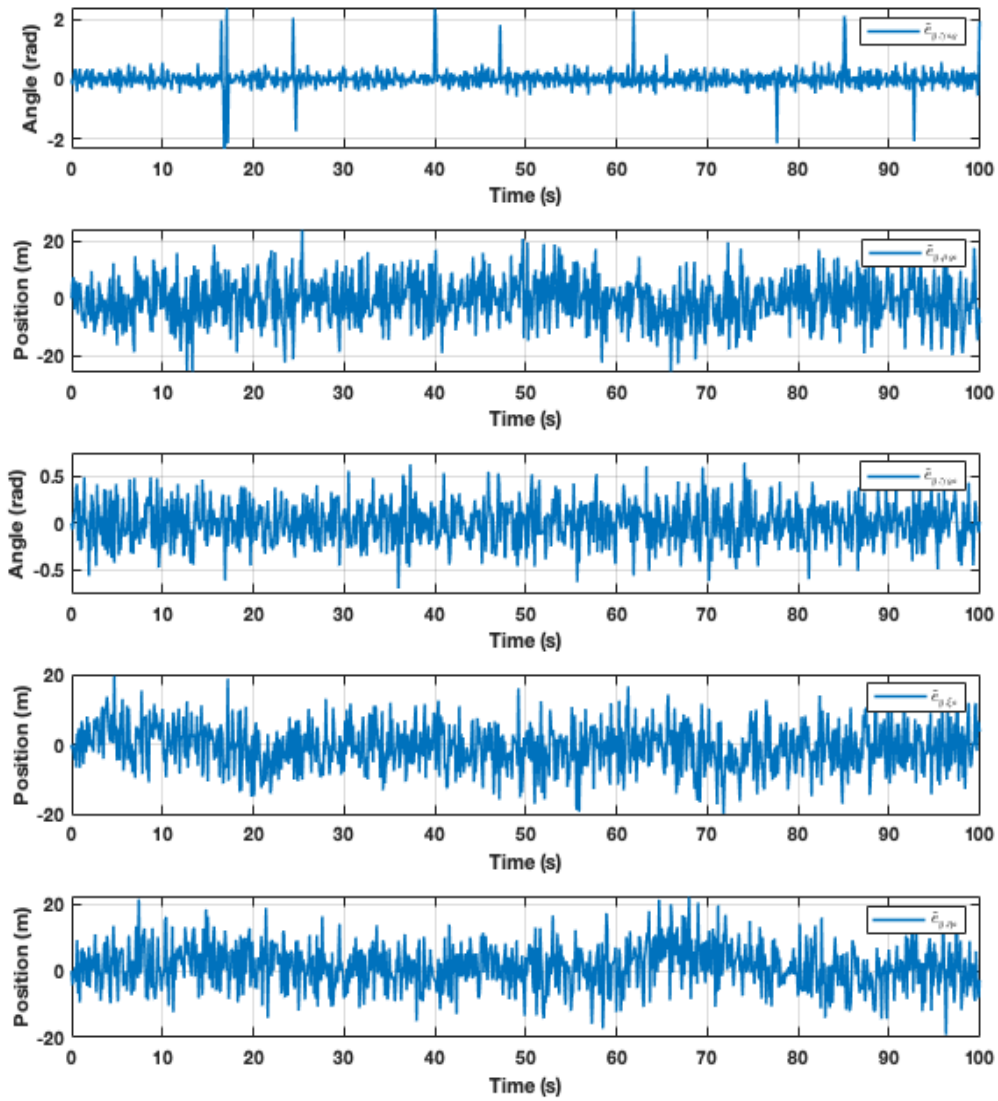

Figure 6: Tuned UKF measurement residuals.

The measurement residuals output from the UKF simulation look like zero mean white noise signals. This is another check on proper functionality of the tuned filter - since error in the simulated sensor observation data is AWGN. The following section discusses tuning parameters and results from Truth Model Testing.

4. Unscented Kalman Filter Truth Model Testing (TMT)

The Unscented KF was tuned in the same iterative manner as in the final report - using NEES and NIS criteria to assess the dynamic performance of the filter. NIS and NEES statistics testing was performed with a significance level of .05. As well, the sigma point generation parameters (α, β) were set to 0.1 and 2, respectively. For the testing, 10 Monte Carlo simulations were computed for the UKF to generate the NEES and NIS statistics for each filter. The number of Monte-Carlo runs for TMT were reduced from the number performed for the LKF and EKF testing because the computation time required for the UKF is significantly longer, due to sigma point propagation. The UKF was initialized with $\hat{\boldsymbol{P}}_0^+ = .9 \cdot diag([5,\ 5,\ 2,\ 20,\ 20,\ 2])$. Figure 8 and Figure 7 show the results from the tuning tests using the following covariance matrices:

$$\boldsymbol{Q}_{UKF} = \begin{bmatrix} 5.0e^{-9} & 0 & 0 & 0 & 0 & 0 \\ 0 & 5.0e^{-9} & 0 & 0 & 0 & 0 \\ 0 & 0 & 5.0e^{-5} & 0 & 0 & 0 \\ 0 & 0 & 0 & 5.0e^{-9} & 0 & 0 \\ 0 & 0 & 0 & 0 & 5.0e^{-9} & 0 \\ 0 & 0 & 0 & 0 & 0 & 5.0e^{-5} \end{bmatrix}$$

$$\boldsymbol{R}_{UKF} = 0.065 \cdot \boldsymbol{R}_{True}$$

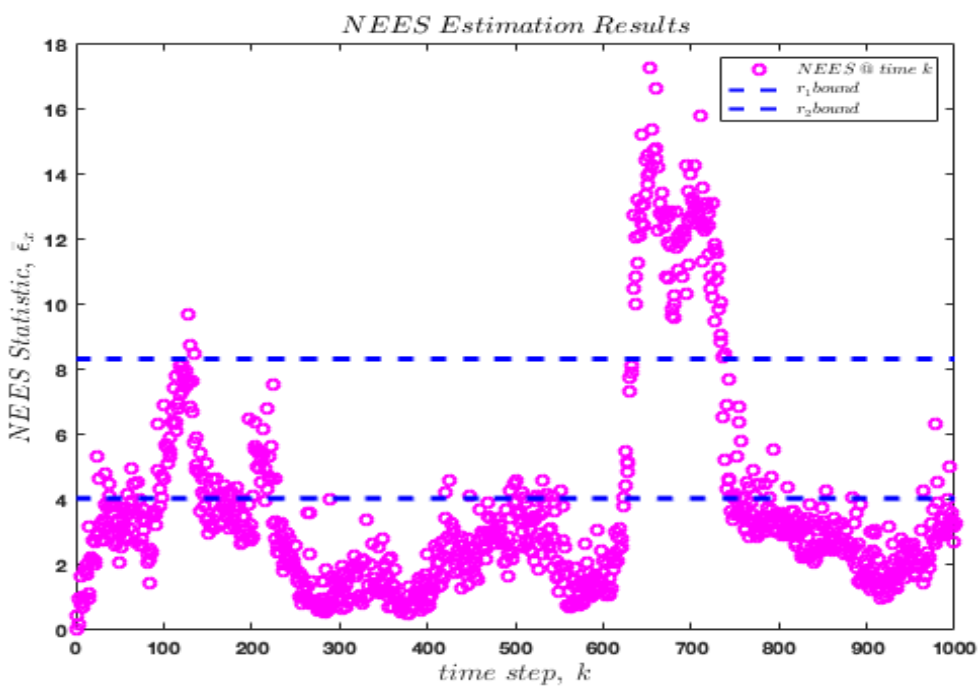

Figure 7: UKF NEES statistics testing results for all time steps. $r_{1,x}$ and $r_{2,x}$ bounds are 4.05 and 8.33.

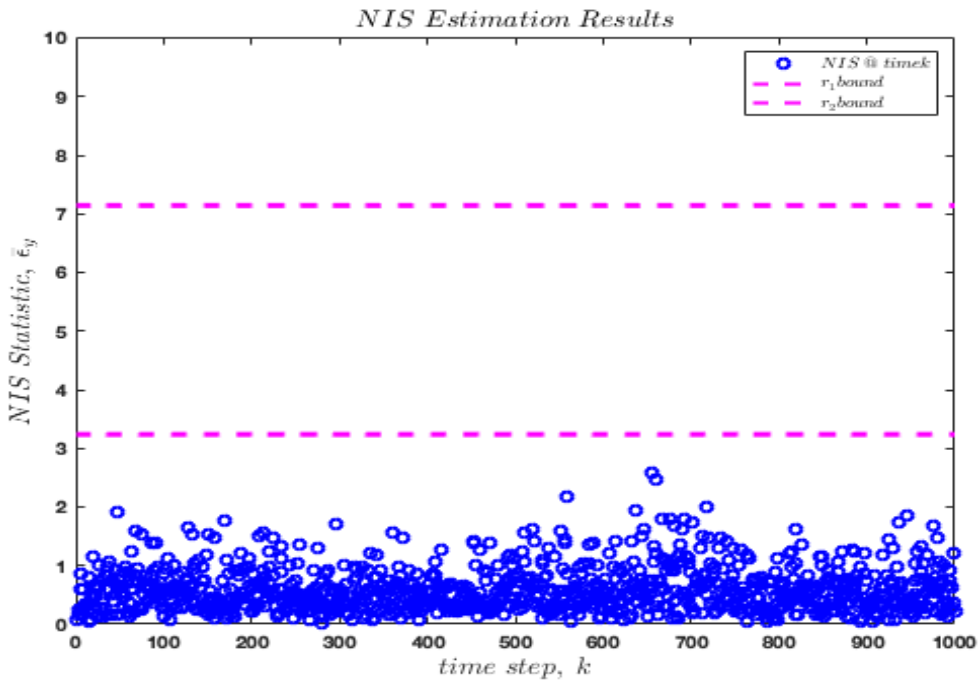

Figure 8: UKF NIS statistics testing results for all time steps. $r_{1,y}$ and $r_{2,y}$ bounds are 3.24 and 7.14

The NEES and NIS tuning test results are not fairly good, but not fantastic. The NEES results are close to meeting the test criteria, but the NIS results are not. Both distributions are centered below the bounds which tells us that the filter is slightly "smug". This is mostly due to computational intensity of Monte-Carlo simulations for the Unscented Kalman Filter, which makes it difficult to tune because of how much time each iteration takes. The for loop used to propagate the the sigma points with $ODE45$ was changed to a $parfor$ loop to reduce run time, but this did not have a large impact on calculation time.

5. Tuned UKF Implementation Results

As was done in the final project, the Unscented KF was implemented using the sensor observation data provided for this project. The estimated states and 2σ values output from the filter are provided below in Figure 9 and Figure 10.

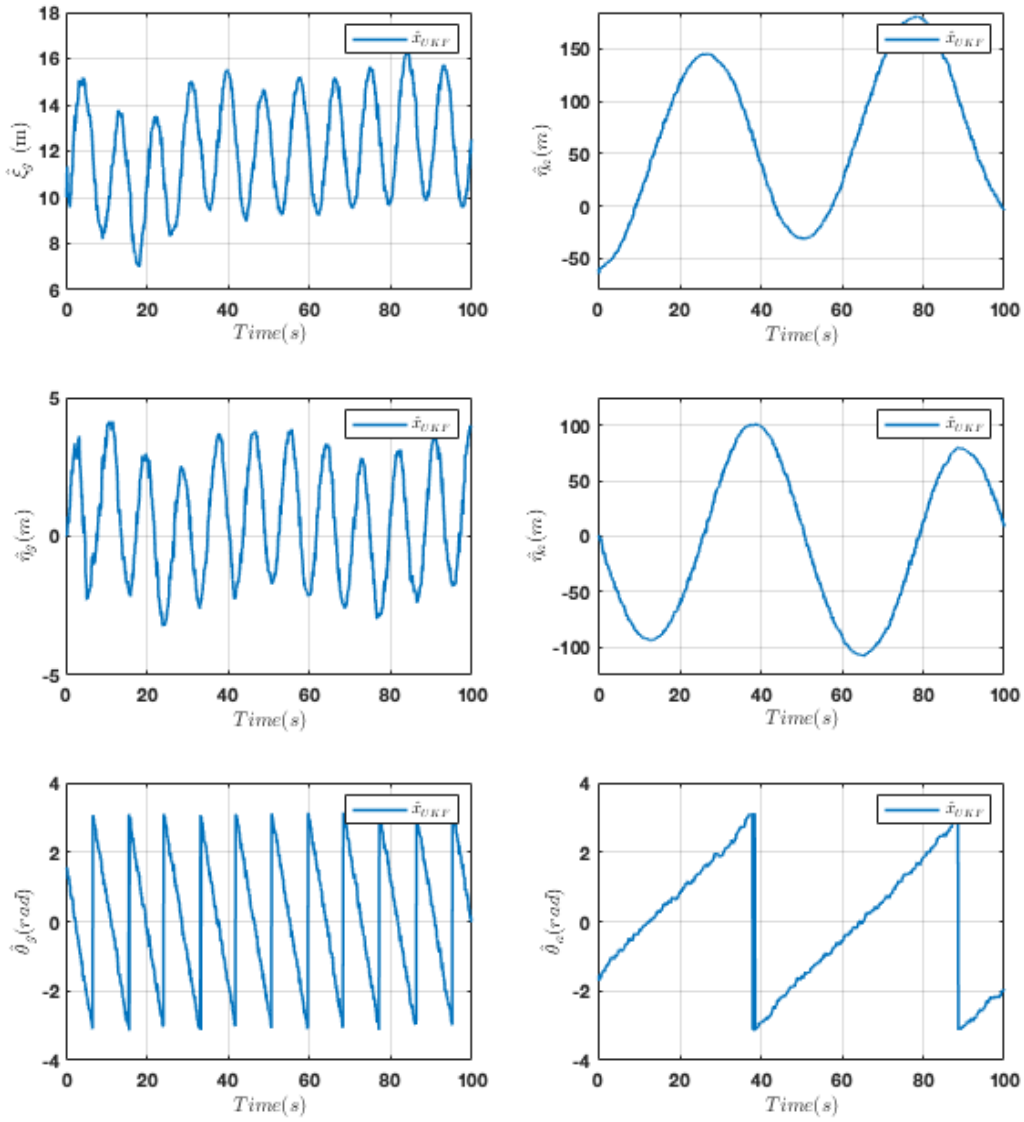

Figure 9: UKF state estimates from provided observation data.

The state estimate results from the UKF look better than both the LKF and EKF - as expected. While estimate results for $\boldsymbol{x}_1$ drift off from the expected trajectory, accuracy has increased in most of the other states. Lastly, it is also interesting to observe the very different shape of this filter's 2σ values compared to those from the EKF and LKF - which use linearized dynamics to compute estimates.

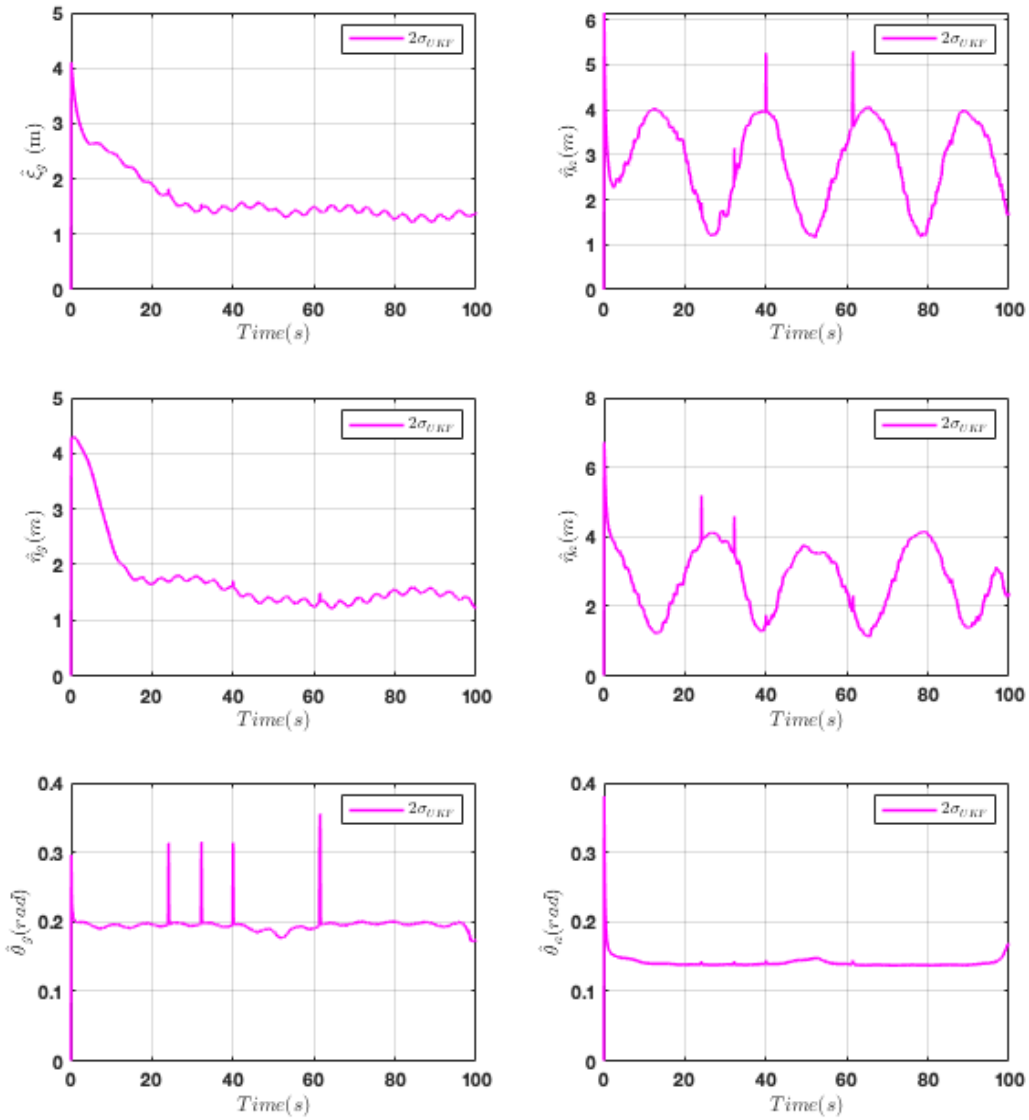

Figure 10: UKF state estimates from provided observation data.

Appendix A. MATLAB Algorithm

```
%% ASEN 5044 Final Project Bonus Question (UKF)

% Henry Dixon
% CU Boulder
% Spring 2020
```

```

% UGV - UAV Robot Cooperative Localization: Unscented KF Implementation

clc; clear; close all;

%% Cooperative System Parameters & Data

%----------------------%
% Nominal Parameters %
%----------------------%

% UGV
L = 0.5; %(m) UGV wheel base length
phig_rng = [-(5*pi)/12,(5*pi)/12]; %(rad) UGV steering angle range
vg_max = 3; %(m/s) max speed
xg_0 = [10; 0; pi/2]; % initial values for UGV states
ug_0 = [2; -pi/18]; % initial values for UGV input states
ug_0L = [2; -5*pi/12]; % lower bound for nominal control values
ug_0U = [3; 5*pi/12]; % upper bound for nominal control values

% UAV
wg_rng = [-pi/6, pi/6]; %(rad/s) turn rate range
va_rng = [10 20]; %(m/s) velocity range
xa_0 = [-60; 0; -pi/2]; % initial values for aircraft states
ua_0 = [12; pi/25]; % initial control input states for aircraft
ua_0L = [12; -pi/6]; % lower bound for nominal control values
ua_0U = [12; pi/6]; % upper bound for nominal control values

% Combined Initial State & Control Values
x0 = [xg_0; xa_0];
u0 = [ug_0; ua_0];
u0_U = [ug_0U;ua_0U];
u0_L = [ug_0L;ua_0L];
mu_x = x0;

% DT Time Step
dt = .1; %(s) time step

% Load Data for Implementing KF and EKF
load('cooplocalization_finalproj_KFdata.mat')

% Measurement & Process Noise Covar Matrices
Q = Qtrue;
R = Rtrue;
ydata1 = ydata;

% Gamma and Omega Matrices
Gamma = eye(6);
Omega = dt*Gamma;
Svx = chol(Q);
Svy = chol(R);
rng(100);

%% Model Simulation & Computation of Nominal Trajectory/Truth Data

```

```
tspan = 0:dt:100; %(s) time span
T = length(tspan);
n = length(x0); % number of states

% State Perturbation Vector & Nonlinear Model Simulation (for comparison to
    reference plots)
perturb_x0 = [0; 0; 0; 0; 0; 0];
x0p = x0 + perturb_x0;
dx = zeros(1,6);
[t,x_nl_check] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,x0p);
x_nl_check(:,3) = wrapToPi(x_nl_check(:,3));
x_nl_check(:,6) = wrapToPi(x_nl_check(:,6));
x_nl_check = x_nl_check'; % transpose states to row vectors

% Generate Ground Truth Data
x_truth = x_nl_check + Svx'*randn(n,T);
ydata = nl_sensor_model_noise(x_truth,R);


%% UKF Simulation

%-------------------------------%
% Run Uncented Kalman Filter %
%-------------------------------%

Qukf = 0.0025*diag([.002 .002 2 .002 .002 2])*Q;
Rukf = 0.065*Rtrue;

ydata(:,1)=[];
mu_0 = x0;
Pkp_ukf = [];
P0 = .9*diag([5 5 2 20 20 2]);
Pkp_ukf(:,1) = reshape(P0,6^2,1);
xhatp_ukf(:,1) = mu_0;

% Sigma Point Parameters
n = length(x0);
alpha = .1;
lambda = alpha^2*n-n;
beta = 2;

% Reset propagation time vector for ODE45
tspan = [];
tspan=0:.1:.1;

for i = 1:length(ydata)

    %-------------------%
    % Prediction Step %
    %-------------------%

    % Compute Sqrt of Covariance
    Pp = reshape(Pkp_ukf(:,i),6,6);
    Sk = chol(Pp);

```

```
    % Generate 2n + 1 Sigma Points
    Xk = sig_points(xhatp_ukf(:,i),Sk,n,lambda);

    % Propagate Sigma Points to k+1
    parfor ii=1:2*n+1
        [~,xnew] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,Xk(:,ii));
        xnew = xnew';
        Xk_m(:,ii)=xnew(:,end);
    end


    % Compute Predicted Mean and Covariance
    xhatm_ukf = zeros(6,1);
    Pkm_ukf = zeros(6,6);

    % Predicted Mean
    for ii = 1:2*n+1
        if ii == 1
            wm = lambda/(lambda+n);
            wc = lambda/(lambda+n)+1-alpha^2+beta;
        else
            wm = 1/(2*(n+lambda));
            wc = wm;
        end
        xhatm_ukf = xhatm_ukf+wm*Xk_m(:,ii);
    end
    xhatm_ukf(3) = wrapToPi(xhatm_ukf(3));
    xhatm_ukf(6) = wrapToPi(xhatm_ukf(6));

    % Predicted Covariance
    for ii = 1:2*n+1
        if ii == 1
            wc = lambda/(lambda+n)+1-alpha^2+beta;
        else
            wc = 1/(2*(n+lambda));
        end
        dxx = Xk(:,ii)-xhatm_ukf;
        dxx(3) = wrapToPi(dxx(3));
        dxx(6) = wrapToPi(dxx(6));
        Pkm_ukf = Pkm_ukf + wc*((dxx)*(dxx)')+Qukf;
    end


    %-------------------------%
    % Measurement Update Step %
    %-------------------------%

    % Compute Sqrt of Predicted Covariance
    Sk=chol(Pkm_ukf);

    % Generate New Sigma Points
    Xk = sig_points(xhatm_ukf,Sk,n,lambda);

    % Compute Nonlinear Sensor Measurements
    Yk = [];
```

```
    for ii = 1:2*n+1
        Yk(:,ii) = nl_sensor_model(Xk(:,ii));
    end

    % Measurement Mean Prediction
    yhatm_ukf = zeros(5,1);
    Pyy = zeros(5,5);
    for ii = 1:2*n+1
        if ii == 1
            wm = lambda/(lambda+n);
        else
            wm = 1/(2*(n+lambda));
        end
        yhatm_ukf = yhatm_ukf+wm*Yk(:,ii);
    end
    yhatm_ukf(1) = wrapToPi(yhatm_ukf(1));
    yhatm_ukf(3) = wrapToPi(yhatm_ukf(3));

    % Measurement Covariance Prediction
    for ii = 1:2*n+1
        if ii == 1
            wc = lambda/(lambda+n)+1-alpha^2+beta;
        else
            wc = 1/(2*(n+lambda));
        end
        dyy = Yk(:,ii)-yhatm_ukf;
        dyy(1) = wrapToPi(dyy(1));
        dyy(3) = wrapToPi(dyy(3));
        Pyy = Pyy + wc*((dyy)*(dyy)')+Rukf;
    end

    % Compute Cross-Covariance Matrix
    Pxy = zeros(6,5);
    for ii = 1:2*n+1
        if ii == 1
            wc = lambda/(lambda+n)+1-alpha^2+beta;
        else
            wc = 1/(2*(n+lambda));
        end
        dyy = Yk(:,ii)-yhatm_ukf;
        dyy(1) = wrapToPi(dyy(1));
        dyy(3) = wrapToPi(dyy(3));
        dxx = Xk(:,ii)-xhatm_ukf;
        dxx(3) = wrapToPi(dxx(3));
        dxx(6) = wrapToPi(dxx(6));
        Pxy = Pxy + wc*((dxx)*(dyy)');
    end

    % Estimate Kalman Gain
    K=Pxy*inv(Pyy);

    % Compute Kalman State and Cov Update with y_k+1
    dely = ydata(:,i) - yhatm_ukf;
    dely(1) = wrapToPi(dely(1)); dely(3) = wrapToPi(dely(3));
    xhat = xhatm_ukf+K*(dely);
```

```
%       xhat(3) = wrapToPi(xhat(3)); xhat(6) = wrapToPi(xhat(6));
    xhatp_ukf(:,1+i) = xhat;

%       dely = ydata(:,i) - yhatm_ukf;
%       dely(1) = wrapToPi(dely(1)); dely(3) = wrapToPi(dely(3));
    yerror(:,i+1) = dely;

    Pp = Pkm_ukf-K*Pyy*K';
    Pkp_ukf(:,i+1) = reshape(Pp,6^2,1);
    sig_ukf(:,i+1) = sqrt(diag(Pp));
end

e_xhat_ukf(:,:) = x_truth(:,:) - xhatp_ukf(:,:);
e_xhat_ukf(3,:) = wrapToPi(e_xhat_ukf(3,:));
e_xhat_ukf(6,:) = wrapToPi(e_xhat_ukf(6,:));
xhatp_ukf(3,:) = wrapToPi(xhatp_ukf(3,:));
xhatp_ukf(6,:) = wrapToPi(xhatp_ukf(6,:));

%-----------------------------------------%
% Plot UKF Estimate Errors & 2sig Bounds %
%-----------------------------------------%
f = 1;
figure(f)
f = f+1;
subplot(611)
plot(tvec,e_xhat_ukf(1,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sqrt(Pkp_ukf(1,:)),'r--','LineWidth',1.5)
plot(tvec,-2*sqrt(Pkp_ukf(1,:)),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\xi_{g,UKF}}$','$2\sigma$','','Interpreter','latex');

subplot(612)
plot(tvec,e_xhat_ukf(2,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sqrt(Pkp_ukf(8,:)),'r--','LineWidth',1.5)
plot(tvec,-2*sqrt(Pkp_ukf(8,:)),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\xi_{g,UKF}}$','$2\sigma$','','Interpreter','latex');

subplot(613)
plot(tvec,e_xhat_ukf(3,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sqrt(Pkp_ukf(15,:)),'r--','LineWidth',1.5)
plot(tvec,-2*sqrt(Pkp_ukf(15,:)),'r--','LineWidth',1.5)
ylim([-1.5*pi, 1.5*pi]);
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\hat{e}_{\xi_{g,UKF}}$','$2\sigma$','','Interpreter','latex');

subplot(614)
plot(tvec,e_xhat_ukf(4,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sqrt(Pkp_ukf(22,:)),'r--','LineWidth',1.5)
plot(tvec,-2*sqrt(Pkp_ukf(22,:)),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
```

```
legend('$\hat{e}_{\xi_{g,UKF}}$','$2\sigma$','','Interpreter','latex');

subplot(615)
plot(tvec,e_xhat_ukf(5,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sqrt(Pkp_ukf(29,:)),'r--','LineWidth',1.5)
plot(tvec,-2*sqrt(Pkp_ukf(29,:)),'r--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\hat{e}_{\xi_{g,UKF}}$','$2\sigma$','','Interpreter','latex');

subplot(616)
plot(tvec,e_xhat_ukf(6,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,2*sqrt(Pkp_ukf(36,:)),'r--','LineWidth',1.5)
plot(tvec,-2*sqrt(Pkp_ukf(36,:)),'r--','LineWidth',1.5)
ylim([-1.5*pi, 1.5*pi]);
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\hat{e}_{\xi_{g,UKF}}$','$2\sigma$','','Interpreter','latex');

%%
%-----------------------------------------%
% Plot UKF Estimates vs  Ground Truth %
%-----------------------------------------%

figure(f)
f = f+1;
subplot(611)
plot(tvec,x_truth(1,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhatp_ukf(1,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\xi_{x,g}$','$\hat{\xi}_{x,g,UKF}$','Interpreter','latex');

subplot(612)
plot(tvec,x_truth(2,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhatp_ukf(2,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\eta_{y,g}$','$\hat{\eta}_{y,g,UKF}$','Interpreter','latex');

subplot(613)
plot(tvec,x_truth(3,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhatp_ukf(3,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\theta_{\omega,g}$','$\hat{\theta}_{\omega,g,UKF}$','Interpreter','
    latex');

subplot(614)
plot(tvec,x_truth(4,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhatp_ukf(4,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\xi_{x,a}$','$\hat{\xi}_{x,a,UKF}$','Interpreter','latex');

```

```
subplot(615)
plot(tvec,x_truth(5,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhatp_ukf(5,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\eta_{y,a}$','$\hat{\eta}_{y,a,UKF}$','Interpreter','latex');

subplot(616)
plot(tvec,x_truth(6,:),'LineWidth',1.5)
grid on; hold on;
plot(tvec,xhatp_ukf(6,:),'--','LineWidth',1.5)
xlabel('Time (s)'); ylabel('Angle(rad)');
legend('$\theta_{\omega,a}$','$\hat{\theta}_{\omega,a,UKF}$','Interpreter','
    latex');

%-----------------------------------%
% Plot UKF Measurement Residuals %
%-----------------------------------%

figure(f)
f = f+1;
subplot(511)
plot(tvec,yerror(1,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\tilde{e}_{y,\gamma_{a,g}}$','Interpreter','latex');

subplot(512)
plot(tvec,yerror(2,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\tilde{e}_{y,\rho_{ga}}$','Interpreter','latex');

subplot(513)
plot(tvec,yerror(3,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Angle (rad)');
legend('$\tilde{e}_{y,\gamma_{g,a}}$','Interpreter','latex');

subplot(514)
plot(tvec,yerror(4,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\tilde{e}_{y,\xi_a}$','Interpreter','latex');

subplot(515)
plot(tvec,yerror(5,:),'LineWidth',1.5)
grid on; hold on;
xlabel('Time (s)'); ylabel('Position (m)');
legend('$\tilde{e}_{y,\eta_a}$','Interpreter','latex');

%%
%-----------------------------------------------------%
% Plot EKF State Estimate & 2 Sigma Bounds %
%-----------------------------------------------------%
```

```
figure(f)
f = f+1;
subplot(321)
plot(tvec,2*sig_ukf(1,:),'m','LineWidth',1.5)
grid on; hold on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\xi}_{g}$ (m)', 'Interpreter','latex');
legend('$2\sigma_{UKF}$','Interpreter','latex');

subplot(323)
plot(tvec,2*sig_ukf(2,:),'m','LineWidth',1.5)
grid on; hold on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\eta}_{g} (m)$','Interpreter','latex');
legend('$2\sigma_{UKF}$','Interpreter','latex');

subplot(325)
plot(tvec,2*sig_ukf(3,:),'m','LineWidth',1.5)
grid on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\theta}_{g} (rad)$','Interpreter','latex');
legend('$2\sigma_{UKF}$','Interpreter','latex');

subplot(322)
plot(tvec,2*sig_ukf(4,:),'m','LineWidth',1.5)
grid on; hold on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\eta}_{a} (m)$','Interpreter','latex');
legend('$2\sigma_{UKF}$','Interpreter','latex');

subplot(324)
plot(tvec,2*sig_ukf(5,:),'m','LineWidth',1.5)
grid on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\eta}_{a} (m)$','Interpreter','latex');
legend('$2\sigma_{UKF}$','Interpreter','latex');

subplot(326)
plot(tvec,2*sig_ukf(6,:),'m','LineWidth',1.5)
grid on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\theta}_{a} (rad)$','Interpreter','latex');
legend('$2\sigma_{UKF}$','Interpreter','latex');

figure(f)
f = f+1;
subplot(321)
plot(tvec,xhatp_ukf(1,:),'LineWidth',1.5)
grid on; hold on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\xi}_{g}$ (m)', 'Interpreter','latex');
legend('$\hat{x}_{UKF}$','Interpreter','latex');

subplot(323)
```

```
plot(tvec,xhatp_ukf(2,:),'LineWidth',1.5)
grid on; hold on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\eta}_{g} (m)$','
    Interpreter','latex');
legend('$\hat{x}_{UKF}$','Interpreter','latex');

subplot(325)
plot(tvec,xhatp_ukf(3,:),'LineWidth',1.5)
grid on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\theta}_{g} (rad)$'
    ,'Interpreter','latex');
legend('$\hat{x}_{UKF}$','Interpreter','latex');

subplot(322)
plot(tvec,xhatp_ukf(4,:),'LineWidth',1.5)
grid on; hold on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\eta}_{a} (m)$','
    Interpreter','latex');
legend('$\hat{x}_{UKF}$','Interpreter','latex');

subplot(324)
plot(tvec,xhatp_ukf(5,:),'LineWidth',1.5)
grid on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\eta}_{a} (m)$','
    Interpreter','latex');
legend('$\hat{x}_{UKF}$','Interpreter','latex');

subplot(326)
plot(tvec,xhatp_ukf(6,:),'LineWidth',1.5)
grid on;
xlabel('$Time (s)$', 'Interpreter','latex'); ylabel('$\hat{\theta}_{a} (rad)$'
    ,'Interpreter','latex');
legend('$\hat{x}_{UKF}$','Interpreter','latex');

%% Truth Model Performance Test (TMT)
% UKF TESTING
%%%***Specify assumed DT noise covariances for KF:
Qkf = 0.0025*diag([.002 .002 2 .002 .002 2])*Q;%diag([10 10 .02 1 1 .02])*Q;
    %%treat this as the main tuning knob
Rkf = 0.065*R; %%assume we can get this from sensor specs
tspan2 = 0:.1:100;
Nsimruns = 10;
NEESsamps = zeros(Nsimruns,length(tvec));
NISsamps = zeros(Nsimruns,length(tvec));
dx = zeros(6,1);
alpha = 0.1;
rng(100);

f=f+1;
for ss=1:Nsimruns
    %%%1. %%Generate true trajectory and measurements from system

    xk_truehist = [];
    ykhist = [];
    xk = x0; %sample initial robot state
```

```
    [t,xk_truehist] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan2,x0);
    xk_truehist = addprocessNoise(xk_truehist',Q);
    xk_truehist=xk_truehist';
    xk_truehist(:,3) = wrapToPi(xk_truehist(:,3));
    xk_truehist(:,6) = wrapToPi(xk_truehist(:,6));
    ynoise=[];

    ynoise=nl_sensor_model_noise(xk_truehist',R);
    ykhist=ynoise';
    ykhist(1,:)=[];
    ykhist=ykhist';

    % UKF time
    mk = x0;
    Pk = .9*diag([5 5 2 20 20 2]);
    Pkp_ukf = [];
    Pkp_ukf = reshape(Pk,6^2,1);
    mk_filt_hist = [];
    Pk_filt_hist = [];
    innovk_hist = [];
    Pyyk_hist = []; %%store measurement innovation covar
    NEESsshist = [];
    NISsshist = [];
    NEESsamps=[];
    NISsamps=[];
    mk_filt_hist(:,1)=mk;
    Pk_filt_hist(:,1)=reshape(Pk,6^2,1);
    dely = [];
    yerror = [];
    tspan = 0:.1:.1;

    for i = 1:length(ykhist)
        %-----------------%
        % Prediction Step %
        %-----------------%

        % Compute Sqrt of Covariance
        Pp = reshape(Pkp_ukf(:,i),6,6);
        Sk = chol(Pp);

        % Generate 2n + 1 Sigma Points
        Xk = sig_points(xhatp_ukf(:,i),Sk,n,lambda);
%         Xk(3,:) = wrapToPi(Xk(3,:));
%         Xk(6,:) = wrapToPi(Xk(6,:));

        % Propagate Sigma Points to k+1
        parfor ii=1:2*n+1
            [~,xnew] = ode45(@(t,x) x_nl_traject_check(x,L,dx,u0),tspan,Xk(:,ii));
            xnew = xnew';
            Xk_m(:,ii)=xnew(:,end);
        end
%         Xk_m(3,:) = wrapToPi(Xk_m(3,:));
%         Xk_m(6,:) = wrapToPi(Xk_m(6,:));

```

```
        % Compute Predicted Mean and Covariance
        xhatm_ukf = zeros(6,1);
        Pkm_ukf = zeros(6,6);

        % Predicted Mean
        for ii = 1:2*n+1
            if ii == 1
                wm = lambda/(lambda+n);
                wc = lambda/(lambda+n)+1-alpha^2+beta;
            else
                wm = 1/(2*(n+lambda));
                wc = wm;
            end
            xhatm_ukf = xhatm_ukf+wm*Xk_m(:,ii);
        end
        xhatm_ukf(3) = wrapToPi(xhatm_ukf(3));
        xhatm_ukf(6) = wrapToPi(xhatm_ukf(6));

        % Predicted Covariance
        for ii = 1:2*n+1
            if ii == 1
                wc = lambda/(lambda+n)+1-alpha^2+beta;
            else
                wc = 1/(2*(n+lambda));
            end
            dxx = Xk(:,ii)-xhatm_ukf;
            dxx(3) = wrapToPi(dxx(3));
            dxx(6) = wrapToPi(dxx(6));
            Pkm_ukf = Pkm_ukf + wc*((dxx)*(dxx)')+Qkf;
%           Pkm_ukf = Pkm_ukf + wc*((Xk_m(:,ii)-xhatm_ukf)*(Xk_m(:,ii)-xhatm_ukf
    )')+Q;
        end


        %-------------------------%
        % Measurement Update Step %
        %-------------------------%

        % Compute Sqrt of Predicted Covariance
        Sk=chol(Pkm_ukf);

        % Generate New Sigma Points
        Xk = sig_points(xhatm_ukf,Sk,n,lambda);
%           Xk(3,:) = wrapToPi(Xk(3,:));
%           Xk(6,:) = wrapToPi(Xk(6,:));

        % Compute Nonlinear Sensor Measurements
        Yk = [];
        for ii = 1:2*n+1
            Yk(:,ii) = nl_sensor_model(Xk(:,ii));
        end
%           Yk(1,:) = wrapToPi(Yk(1,:));
%           Yk(3,:) = wrapToPi(Yk(3,:));

        % Measurement Mean Prediction
```

```
        yhatm_ukf = zeros(5,1);
        Pyy = zeros(5,5);
        for ii = 1:2*n+1
            if ii == 1
                wm = lambda/(lambda+n);
            else
                wm = 1/(2*(n+lambda));
            end
            yhatm_ukf = yhatm_ukf+wm*Yk(:,ii);
        end
        yhatm_ukf(1) = wrapToPi(yhatm_ukf(1));
        yhatm_ukf(3) = wrapToPi(yhatm_ukf(3));

        % Measurement Covariance Prediction
        for ii = 1:2*n+1
            if ii == 1
                wc = lambda/(lambda+n)+1-alpha^2+beta;
            else
                wc = 1/(2*(n+lambda));
            end
            dyy = Yk(:,ii)-yhatm_ukf;
            dyy(1) = wrapToPi(dyy(1));
            dyy(3) = wrapToPi(dyy(3));
            Pyy = Pyy + wc*((dyy)*(dyy)')+Rkf;
%           Pyy = Pyy + wc*((Yk(:,ii)-yhatm_ukf)*(Yk(:,ii)-yhatm_ukf)')+R;
        end

        % Compute Cross-Covariance Matrix
        Pxy = zeros(6,5);
        for ii = 1:2*n+1
            if ii == 1
                wc = lambda/(lambda+n)+1-alpha^2+beta;
            else
                wc = 1/(2*(n+lambda));
            end
            dyy = Yk(:,ii)-yhatm_ukf;
            dyy(1) = wrapToPi(dyy(1));
            dyy(3) = wrapToPi(dyy(3));
            dxx = Xk(:,ii)-xhatm_ukf;
            dxx(3) = wrapToPi(dxx(3));
            dxx(6) = wrapToPi(dxx(6));
            Pxy = Pxy + wc*((dxx)*(dyy)');
%           Pxy = Pxy + wc*((Xk(:,ii)-xhatm_ukf)*(Yk(:,ii)-yhatm_ukf)');
        end

        % Estimate Kalman Gain
        K=Pxy*inv(Pyy);

        % Compute Kalman State and Cov Update with y_k+1
        dely = ydata(:,i) - yhatm_ukf;
        dely(1) = wrapToPi(dely(1)); dely(3) = wrapToPi(dely(3));
        xhat = xhatm_ukf+K*(dely);
        mk_filt_hist(:,i+1) = xhat;
%         xhat(3) = wrapToPi(xhat(3)); xhat(6) = wrapToPi(xhat(6));
        xhatp_ukf(:,i+1) = xhat;
```

```
        yerror(:,i+1) = dely;

        Pp = Pkm_ukf-K*Pyy*K';
        Pkp_ukf(:,i+1) = reshape(Pp,6^2,1);
        Pk_filt_hist(:,i+1) = reshape(Pp,6^2,1);

        %%Compute and store NEES and NIS statistics:
        innov_kp1 = yerror(:,i+1);
        invPkp1 = inv(Pp);
        invPyykp1 = inv(Pyy);
        NEESsshist(i) = ...
            (xk_truehist(i,:)' - mk_filt_hist(:,i))'*invPkp1*(xk_truehist(i,:)
    ' - mk_filt_hist(:,i));
        NISsshist(i) = innov_kp1'*invPyykp1*innov_kp1;
    end

    NEESsamps(ss,:) = NEESsshist;
    NISsamps(ss,:) = NISsshist;
     %%sanity check:
    mk_filt_hist(3,:) = wrapToPi(mk_filt_hist(3,:));
    mk_filt_hist(6,:) = wrapToPi(mk_filt_hist(6,:));

    %Plot state estimation errors versus time
    figure(f),
    clf
    subplot(611)
    plot(tvec,xk_truehist(:,1)-mk_filt_hist(1,:)','b','LineWidth',2), hold on
    plot(tvec,2*sqrt((Pk_filt_hist(1,:))),'b--','LineWidth',1)
    plot(tvec,-2*sqrt((Pk_filt_hist(1,:))),'b--','LineWidth',1)
    subplot(612)
    plot(tvec,xk_truehist(:,2)-mk_filt_hist(2,:)','b','LineWidth',2), hold on
    plot(tvec,2*sqrt((Pk_filt_hist(8,:))),'b--','LineWidth',1)
    plot(tvec,-2*sqrt((Pk_filt_hist(8,:))),'b--','LineWidth',1)
    subplot(613)
    plot(tvec,xk_truehist(:,3)-mk_filt_hist(3,:)','b','LineWidth',2), hold on
    plot(tvec,2*sqrt((Pk_filt_hist(15,:))),'b--','LineWidth',1)
    plot(tvec,-2*sqrt((Pk_filt_hist(15,:))),'b--','LineWidth',1)
    subplot(614)
    plot(tvec,xk_truehist(:,4)-mk_filt_hist(4,:)','b','LineWidth',2), hold on
    plot(tvec,2*sqrt((Pk_filt_hist(22,:))),'b--','LineWidth',1)
    plot(tvec,-2*sqrt((Pk_filt_hist(22,:))),'b--','LineWidth',1)
    subplot(615)
    plot(tvec,xk_truehist(:,5)-mk_filt_hist(5,:)','b','LineWidth',2), hold on
    plot(tvec,2*sqrt((Pk_filt_hist(29,:))),'b--','LineWidth',1)
    plot(tvec,-2*sqrt((Pk_filt_hist(29,:))),'b--','LineWidth',1)
    subplot(616)
    plot(tvec,xk_truehist(:,6)-mk_filt_hist(6,:)','b','LineWidth',2), hold on
    plot(tvec,2*sqrt((Pk_filt_hist(36,:))),'b--','LineWidth',1)
    plot(tvec,-2*sqrt((Pk_filt_hist(36,:))),'b--','LineWidth',1)
end


%%DO NEES TEST:
epsNEESbar = mean(NEESsamps,1);
```

```
alphaNEES = 0.05; %%significance level
Nnx = Nsimruns*n;
%%compute intervals:
r1x = chi2inv(alphaNEES/2, Nnx )./ Nsimruns
r2x = chi2inv(1-alphaNEES/2, Nnx )./ Nsimruns

f=f+1;
figure(f)
plot(epsNEESbar,'mo','MarkerSize',6,'LineWidth',2),hold on
plot(r1x*ones(size(epsNEESbar)),'b--','LineWidth',2)
plot(r2x*ones(size(epsNEESbar)),'b--','LineWidth',2)
ylabel('$NEES \; Statistic, \; \bar{\epsilon}_{x}$','FontSize',14,'Interpreter','latex')
xlabel('$time \; step, \; k$','FontSize',14,'Interpreter','latex')
title('$NEES \; Estimation \; Results$','FontSize',14,'Interpreter','latex')
legend('$NEES \; @ \; time \; k$', '$r_1 bound$', '$r_2 bound$','Interpreter','latex')
axis([0 1000 r1x-5 r2x+5])
axis([0 1000 0 20])
%%DO NIS TEST:
epsNISbar = mean(NISsamps,1);
alphaNIS = 0.05;
Nny = Nsimruns*length(ydata(:,2));
%%compute intervals:
r1y = chi2inv(alphaNIS/2, Nny )./ Nsimruns
r2y = chi2inv(1-alphaNIS/2, Nny )./ Nsimruns
f=f+1;
figure(f)
plot(epsNISbar,'bo','MarkerSize',6,'LineWidth',2),hold on
plot(r1y*ones(size(epsNISbar)),'m--','LineWidth',2)
plot(r2y*ones(size(epsNISbar)),'m--','LineWidth',2)
ylabel('$NIS \; Statistic, \; \bar{\epsilon}_y$','FontSize',14,'Interpreter','latex')
xlabel('$time \; step,\; k$','FontSize',14,'Interpreter','latex')
title('$NIS \; Estimation \; Results$','FontSize',14,'Interpreter','latex')
legend('$NIS \; @ \; time k$', '$r_1 bound$', '$r_2 bound$','Interpreter','latex')
axis([0 1000 0 15])


%% Function Definitions

function xdot_nl = x_nl_traject(x,L,u,u0,t)
% Combined UGV & UAV Nonlinear Dynamics with control forcing input
u = u(t)+u0;

xdot_nl = [u(1)*cos(wrapToPi(x(3))); u(1)*sin(wrapToPi(x(3))); (u(1)/L)*tan(u(2));...
    u(3)*cos(wrapToPi(x(6))); u(3)*sin(wrapToPi(x(6))); u(4)];
end

function xdot_nl_check = x_nl_traject_check(x,L,dx,u)
% Combined UGV & UAV Nonlinear Dynamics with x-perturbation input
delx = [dx(1); dx(2); dx(3); dx(4); dx(5); dx(6)];

```

```
xdot_nl = [u(1)*cos(wrapToPi(x(3))); u(1)*sin(wrapToPi(x(3))); (u(1)/L)*tan(u(2));...
    u(3)*cos(wrapToPi(x(6))); u(3)*sin(wrapToPi(x(6))); u(4)] + delx;

xdot_nl_check = [xdot_nl(1); xdot_nl(2); wrapToPi(xdot_nl(3));...
    xdot_nl(4); xdot_nl(5); wrapToPi(xdot_nl(6))];
end

function y_obs = nl_sensor_model(x)
% Nonlinear Sensor Dynamics Model

y_obs(1,:) = wrapToPi(atan2((x(5,:)-x(2,:)),(x(4,:)-x(1,:)))-x(3,:));
y_obs(2,:) = sqrt((x(1,:)-x(4,:)).^2+(x(2,:)-x(5,:)).^2);
y_obs(3,:) = wrapToPi(atan2((x(2,:)-x(5,:)),(x(1,:)-x(4,:)))-x(6,:));
y_obs(4,:) = x(4,:);
y_obs(5,:) = x(5,:);

end

function y_obs = nl_sensor_model_noise(x,R)
% Nonlinear Sensor Dynamics Model

y_obs(1,:) = wrapToPi(atan2((x(5,:)-x(2,:)),(x(4,:)-x(1,:)))-x(3,:))+sqrt(R(1,1))*randn(1,length(x));
y_obs(2,:) = sqrt((x(1,:)-x(4,:)).^2+(x(2,:)-x(5,:)).^2)+sqrt(R(2,2))*randn(1,length(x));
y_obs(3,:) = wrapToPi(atan2((x(2,:)-x(5,:)),(x(1,:)-x(4,:)))-x(6,:))+sqrt(R(3,3))*randn(1,length(x));
y_obs(4,:) = x(4,:)+sqrt(R(4,4))*randn(1,length(x));
y_obs(5,:) = x(5,:)+sqrt(R(5,5))*randn(1,length(x));

end

function [A,B,F,G,H] = matrixUpdate(x,u,dt,L)
% Update Linearized CT Matrices and Discretize

    % A: Linearized Plant Dynamics Matrix
    A = [0 0 -u(1)*sin(x(3)) 0 0 0;...
        0 0 u(1)*cos(x(3)) 0 0 0;
        0 0 0 0 0 0;...
        0 0 0 0 0 -u(3)*sin(x(6));...
        0 0 0 0 0 u(3)*cos(x(6));...
        0 0 0 0 0 0];

    % B: Linearized Control Input Matrix
    B = [cos(x(3)) 0 0 0;...
        sin(x(3)) 0 0 0;...
        (tan(u(2))/L) ((u(1)/L)*(sec(u(2))^2)) 0 0;...
        0 0 cos(x(6)) 0;...
        0 0 sin(x(6)) 0;...
        0 0 0 1];

    % H: Linearized Sensor Matrix
    H(1,1) = (x(5)-x(2))/(((x(5)-x(2))^2/(x(4)-x(1))^2+1)*(x(4)-x(1))^2);
    H(1,2) = -1/(((x(5)-x(2))^2/(x(4)-x(1))^2+1)*(x(4)-x(1)));
```

```
    H(1,3) = -1;
    H(1,4) = -H(1,1);
    H(1,5) = -H(1,2);
    H(1,6) = 0;
    H(2,1) = (x(1)-x(4))/sqrt((x(1)-x(4))^2+(x(2)-x(5))^2);
    H(2,2) = (x(2)-x(5))/sqrt((x(1)-x(4))^2+(x(2)-x(5))^2);
    H(2,4) = -H(2,1);
    H(2,5) = -H(2,2);
    H(3,1) = -(x(2)-x(5))/(((x(2)-x(5))^2/(x(1)-x(4))^2+1)*(x(1)-x(4))^2);
    H(3,2) = 1/(((x(2)-x(5))^2/(x(1)-x(4))^2+1)*(x(1)-x(4)));
    H(3,4) = -H(3,1);
    H(3,5) = -H(3,2);
    H(3,6) = -1;
    H(4,4) = 1;
    H(5,5) = 1;

    % F & G: Linearized & Discretized State Matrices
    F = eye(6) + dt*A;
    G = dt*B;
end
function xnoise = addprocessNoise(X,Q)
for i=1:length(X)
    X(1,i)=X(1,i)+1*sqrt(Q(1,1))*randn(1);
    X(2,i)=X(2,i)+1*sqrt(Q(2,2))*randn(1);
    X(3,i)=X(3,i)+1*sqrt(Q(3,3))*randn(1);
    X(4,i)=X(4,i)+1*sqrt(Q(4,4))*randn(1);
    X(5,i)=X(5,i)+1*sqrt(Q(5,5))*randn(1);
    X(6,i)=X(6,i)+1*sqrt(Q(6,6))*randn(1);
end
xnoise=X;
end

% Sigma Point Generator
function Xk = sig_points(xhat_p,Sk,n,lambda)
% Function to compute sigma points for UKF

Xk = [xhat_p xhat_p+sqrt(n+lambda)*(Sk(1,:))' xhat_p+sqrt(n+lambda)*(Sk(2,:))'...
        xhat_p+sqrt(n+lambda)*(Sk(3,:))' xhat_p+sqrt(n+lambda)*(Sk(4,:))'...
        xhat_p+sqrt(n+lambda)*(Sk(5,:))' xhat_p+sqrt(n+lambda)*(Sk(6,:))' ...
        xhat_p-sqrt(n+lambda)*(Sk(1,:))' xhat_p-sqrt(n+lambda)*(Sk(2,:))'...
        xhat_p-sqrt(n+lambda)*(Sk(3,:))' xhat_p-sqrt(n+lambda)*(Sk(4,:))'...
        xhat_p-sqrt(n+lambda)*(Sk(5,:))' xhat_p-sqrt(n+lambda)*(Sk(6,:))'];

end
```